

Formulario C++ Básico

Estructura básica

```
#include <iostream>
#include "iostream"
using namespace std;
int main() {
    cout << "Hola";
    return 0;
}
```

La diferencia entre

`#include <iostream>` e `#include "filename"`

es la forma en la que el compilador busca los archivos. Con `< >` el compilador busca el archivo en una dirección estandar, mientras que con `" "`, el archivo se busca primero en el directorio del proyecto. Si no se encuentra ahí, entonces lo busca como con `< >`.

Tipos de Datos

Type	Size (bytes)	Ejemplo	Descripción
int	4 bytes	int x = 10	Números enteros
char	1 byte	char c = 'A'	Un sólo caracter
short	2 bytes	short s = 100	Entero pequeño
long	4-8 bytes	long l = 100000	Entero grande
float	4 bytes	float f = 10.5	Decimal Single-precision
double	8 bytes	double d = 10.55	Decimal Double-precision
bool	1 byte	bool b = true	True/False Values
pointer	4-8 bytes	int * p = &v ('v' being an int type)	Guarda dirección de memoria de una variable
void	-	-	Representa la falta de valor

Operaciones Aritméticas

Operación	Símbolo	Ejemplo	Asignación Compuesta
Suma	+	a + b	a+=b → a = a+b
Resta	-	a - b	a-=b → a = a-b
Multiplicación	*	a * b	a*=b → a = a*b
División	/	a / b	a/=b → a = a/b
Modulo	%	a % b	a%=b → a = a%b
Potencia	NA	<cmath> std::pow(a, 3)	NA

Comparadores

Operador	Descripción
==	Igual a
!=	NO igual a
<	Menor que
>	Mayor que
<=	Menor o igual que
>=	Mayor o igual que

Operaciones lógicas

Operaciones lógicas BITWISE (Sólo variables enteras)

Operador	Operación lógica	Ejemplo a 8 bits
&	Bitwise AND	00001100 (12) & 00011001 (25) = 00001000 (8)
	Bitwise OR	00001100 (12) 00011001 (25) = 00011101 (29)
^	Bitwise XOR	00001100 (12) ^ 00011001 (25) = 00010101 (21)
~	Bitwise Complemento	~ 00001100 (12) = 11110011 (-13)
<<	Left Shift	00001100 (12) << 1 = 00011000 (24)
>>	Right Shift	00001100 (12) >> 1 = 00001100 (6)

Con estos operadores también se puede hacer la Asignación Compuesta.

Por sí solo, C++ no permite usar estos operadores con floats o doubles, pero se puede hacer utilizando

```
std::bitcast (C++ 20, <bit>)
```

para obtener la representación binaria del número real y poder usar los operadores de BITWISE (No es común, pero se puede).

Control

```
if (COND) {}  
for (int i=0;i<n;i++) {}  
while(COND) {}
```

Funciones

```
int suma(int a, int b) {  
    return a + b;  
}
```

Clases

```
class A {  
public:  
    int x;  
    A(int v): x(v) {}  
};
```

STL

- vector<T>
- map<K,V>
- set<T>
- unordered_map

Compilación

- g++ main.cpp -o main
- g++ -std=c++17 file.cpp